

注意事项:

- (1) 本试卷不是按照题目难度排序。题量较大，注意合理安排时间。
- (2) 采用本课程教材和课件的统一前提假设，包括但不限于：x86-64/Linux；System V AMD64 ABI；AT&T 语法；Y86-64；SEQ/PIPE 处理器。现实中有多种解读的，以本课程为准。
- (3) 在答题纸上答题，写明大题和小题的题号。
- (4) 选择题的回答按行排布，画好表格，每 10 题换行。
- (5) 不定项选择题每题 3 分，少选得 1 分，错选得 0 分。题目中出现类似“哪些”的描述，不代表一定会有多个答案。

一 不定项选择题（上）（39 分）

(信息的表示和处理)

1. 观察下面的 C 程序代码:

答案: A, B, D, E

(x 与 u 比较时会把 x 转成无符号; u+50 发生模 (2^{32}) 回绕)

2. 对任意非负 int x, 执行:

答案: A, B, D

(float 只有 24 位有效精度; 超过 (2^{24}) 后相邻可表示整数间隔变大, 可能向上舍入)

3. 对任意 unsigned int x, 定义:

答案: A, B, C, D

(清零低 5 位; E 错在不是 +1 而是减去低 5 位形成的余数)

4. 在 x86-64 常见对齐规则下, 给定:

答案: A, B, D, E

(d 需要 8 对齐, 导致 s 后面填充到 16; 重排可显著减少填充)

(程序的机器级表示)

5. 观察下面的汇编代码:

答案: ACD

解析: addl 是 32 位加法: %eax=0xFFFFFFFF + 1 -> 0x00000000 (低 32 位回绕)。写 %eax 会把 %rax 的高 32 位清零, 所以 %rax 变为全 0。回绕产生进位所以 CF=1; 结果为 0 所以 ZF=1; 有符号角度是 $-1 + 1 = 0$ 不溢出, OF=0。

6. 观察下面的汇编代码片段:

答案: ABDE

解析: push %rbp; mov %rsp,%rbp 后, %rbp 指向保存的旧 %rbp, 其上方 8 字节是返回地址。入口通常 8(mod16), push 后变 0(mod16), 再 sub 48 仍为 0(mod16),

所以 C 错。leave 语义是 `mov %rbp,%rsp; pop %rbp`，在该帧结构下等价于先把 `%rsp` 加回 48 再弹出 `%rbp`。

7. 调用一个有 8 个 long 参数的函数 `f(a1,...,a8)`，调用点使用如下汇编代码序列

答案: ABC

解析: 超过 6 个整数参数后，其余从栈上传递；在被调函数入口，`0(%rsp)` 是返回地址，因此第 7 个参数在 `8(%rsp)`，第 8 个在 `16(%rsp)`。caller 负责把自己 push 的参数空间收回。`%r10/%r11` 不是传参寄存器；是否用 `%rbp` 是编译器选择，不是 ABI 强制。

(处理器体系结构)

8. 在 Y86-64 指令集中，下列哪些指令在指令字节序列中同时包含 `regids` 字节与 `valC` 常量字段？

答案: AB

解析: `irmovq/rmmovq/mrmovq` 需要寄存器说明 (`regids`) 且需要常量/位移 (`valC`)。`call/jXX` 只有 `valC`；`OPq` 只有 `regids`；`ret` 两者都不需要。

9. 关于 `ret` 在 PIPE 中引发的控制冒险，下列说法哪些正确？

答案: ABDE

解析: `ret` 的返回地址要等到读内存后才知道；因此常见做法是暂停取指并用 bubble 填充前端，直到返回地址可用，再用 `W_valM` 改道 PC。它不是普通的寄存器 RAW 冒险，不能靠 forwarding“提前变出”目标 PC。

(存储器层次结构+优化程序性能)

10. 关于“局部性 (locality)”，下列说法哪些正确？

答案: ABC

解析: A/B/C 都是教材强调的经典现象；D 反了 (行优先按行更连续)；E 错在 block 变大主要就是利用空间局部性 (一次带回相邻数据)。

11. 关于高速缓存的写策略，下列说法哪些正确？

答案: AB

解析: C/D/E 都是把概念说反或忽略现实代价: `write-allocate` 是“写 miss 先装入再写”；`write-back` 需要管理不同情况，控制更复杂；`write-through` 往往更需要 `write buffer` 以免写操作拖慢流水。

12. 在一个有两级高速缓存 (cache) 的计算机系统里，设:

答案: ABCE

解析: D 反了: L2 更少 miss 会降低 L1 的平均 miss penalty，从而降低平均访存时间

13. 关于教材里的 memory mountain (吞吐/带宽随工作集大小与 stride 变化) 的解释，下列说法哪些正确？

答案: ABD

解析: `stride=1` 的主要收益是空间局部性而不是“反复访问同一元素”(C 错); E 明显错误。

二 不定项选择题 (下) (36 分)

(链接)

14. 三个 C 程序文件, 下面仅观察其中的全局符号 `g`:

答案: ACD

解析: `int g=1` 是强定义; `int g;` 是“弱/共同 (common)”风格, 强胜弱, 因此 A 对、B 错。两个强定义会报错 (D 对)。链接器通常不做类型一致性检查, E 错 (但运行行为会出问题)。

(异常控制流+系统级 I/O)

15. 关于异常 (exception) 的细分类型, 下列说法哪些正确?

答案: ABD

解析: `trap` 常见于系统调用, 返回到下一条; `fault` 典型会重启当前指令 (例如页故障修好后重新执行触发缺页的那条访存指令), 因此 C 错; `page fault` 是 `fault`, 不是 `trap` (E 错)。

16. 关于“进程上下文切换 (context switch)”, 下列说法哪些正确?

答案: ACD

解析: B 错 (不一定, 有可能还在当前进程); E 错 (切换进程通常意味着切换到另一个地址空间/页表)。

17. 文件 `t.txt` 的内容恰好是 4 字节: `abcd` (示意)

答案: ABE

解析: 父子进程共享同一个 `open file description`, 因此 4 次 `read` 总体按顺序消耗 `a, b, c, d`。某个进程读到的两字节一定是“先后顺序递增”的组合 (如 `ab/ac/bd/cd/...`), 但不可能倒序 (`ca`) 或重复 (`aa`)。

18. 观察下面的 C 程序代码

答案: AB

解析: `SIGCHLD` 可能合并递送; 一次进入 `handler` 用 `while(waitpid...)` 可回收多个已退出子进程, 因此 A 对。`reap==2` 意味着两个子都已被回收, 通常不会有 `zombie` (B 对)。C/D/E 都不成立。

19. 观察下面的 C 程序代码:

答案: B

解析: 普通信号 (非实时信号) 通常不排队: 阻塞期间同种信号 `pending` 只记一次, 因此解除阻塞后 `handler` 只会跑一次 (B/E 对), 解除阻塞前不会递送 (D 错)。

20. 关于 `execve()` 成功执行后的语义, 下列说法哪些正确?

答案: ABD

解析: `exec` 是“在同一进程里换程序”, `PID` 不变 (C 错), 也不会创建新进程 (E 错)

(虚拟存储)

21. 执行下面的 C 程序代码

答案: BE

解析: `fork` 后父子地址空间逻辑上独立, 但常用 COW 让它们先共享物理页; 一方写入时再复制, 因此父仍读到 10, 而子写可能触发一次“为了复制而产生”的 `fault`。改成共享匿名映射则写入可见。

22. 关于 TLB, 下列说法哪些正确?

答案: AB

解析: TLB 是“页表项缓存”; TLB miss 会引发页表遍历甚至缺页处理)。上下文切换时是否“必须清空”取决于是否有地址空间标识等机制, 因此 C 说法太绝对。TLB 命中时, 如果有效位为 0, 也会导致缺页 (E 错误)

23. 关于动态内存分配, 下列说法哪些正确?

答案: ABE

解析: CD 描述反了

(并发+网络编程)

24. 并发网络服务器的这段程序代码:

答案: AC

解析: `fork` 后两边都持有 `listenfd/connfd`, 子进程应关 `listenfd` 防泄漏; 父进程应回收子进程避免僵尸。父进程关 `connfd` 不影响子进程持有的 `connfd` (D 错)。E 错误, 网络服务要一直运行。

25. 关于进程 (processes) 和线程 (threads) 的描述, 下列说法哪些正确?

答案: ABCD

解析: 线程切换会保存 CPU 的寄存器, E 错误

三 链接相关主题 (10 分)

1. 符号表分析 (7 分) 每行 1 分

	符号	.symtab 有条目?	符号类型	定义符号的模块	节或伪节名
1	<code>arr</code>	是	全局	<code>main.o</code>	<code>.data</code>
2	<code>result</code>	是	全局	<code>main.o</code>	<code>.bss</code>
3	<code>fix</code>	是	局部	<code>main.o</code>	<code>.data</code>
4	<code>sum_array</code>	是	外部	<code>sum.o</code>	<code>.text</code>
5	<code>i</code>	否	/	/	/
6	<code>sum</code>	否	/	/	/
7	<code>printf</code>	是	外部	<code>libc.so</code>	<code>.text</code>

2. 重定位分析 (2 分)

已知以下重定位条目信息。请根据重定位信息填写空白处的机器码 (32 位小端编码)。

(注: 在答题纸上画出下面表格的第 1 列和第 3 列)

编号	重定位条目信息	重定位引用值 32 位编码 (参照代码中的 16 进制形式)
①	r.offset = 0x09 r.symbol = arr r.type = R_X86_64_32 r.addend = 0	10 20 40 00
②	r.offset = 0x0e r.symbol = sum_array r.type = R_X86_64_PC32 r.addend = -4	25 00 00 00

3. 链接顺序影响 (1 分)

……以下说法正确的是 (B)

- A) 两者大小相同, 内容相同 B) 两者大小相同, 内容不同
C) 两者大小不同, 内容相同 D) 两者大小不同, 内容不同

四 并发相关主题 (15 分)

1. (4 分) 以下是一段改编自教材上的多线程代码。对于 thread 函数内出现的以下 8 个变量: i, niters, local_thread, static_local, cnt, index, shared_array, dynamic_ptr, 哪几个是共享变量?

答:

答案: static_local; cnt; shared_array; dynamic_ptr (每个 1 分)

2. (3 分)

(1) 交换 reader 函数里的 lock(&reader_switch, &wmutex); 和 V(&empty); 两行, 是否会导致竞争或死锁问题? 答: () [是/否]

答案: 否 (1 分)

(2) 在本题的 reader 和 writer 实现基础上, ……

答: ()

- A. (9), (8), (4), (2), (11)
B. (11), (8), (3), (3), (8)
C. (11), (11), (3), (2), (8)
D. (9), (11), (4), (3), (11)

答案: C (2 分)

3. (8 分) 在现代餐饮行业中, ……答案 (每项 1 分):

- (1) 1
(2) BUF_SIZE
(3) P(&buffer.empty)
(4) P(&buffer.mutex)
(5) V(&buffer.full)
(6) P(&buffer.full)
(7) P(&buffer.mutex)
(8) V(&buffer.empty)